University of Massachusetts Amherst

Department of Microbiology[1]

Department of Electrical and Computer Engineering[2]

2D Skeletonization

Notebook Documentation

Version 1.0 · May 2026

Prepared by: Cooper Richman[2]

Supervisors

Dr. Sang Hyun Lee[1]

Dr. Bill Leonard[2]

Contents

Chapter 1

Introduction

The **2D Skeletonization** notebook (`2D Skeletonization.ipynb`) provides a set of independent Python cells for generating and visualizing two-dimensional skeletal representations of mycelial fungal networks. Inputs may be cleaned `.vtk` mesh files or raw `.nd2` microscopy image files. In either case, the data is reduced to a 2D binary occupancy map via a Z-axis max projection, after which morphological skeletonization is applied to extract the 1-pixel-wide centerline structure.

The resulting skeletons are used for preliminary inspection of network topology and serve as the precursor binary image stacks ingested by the **Mycelial Path Viewer & Export Tool**.

1.1 Key Capabilities

- Z-axis max projection of 3D voxel grids to produce 2D binary masks.
- Morphological 2D skeletonization via `skimage.morphology.skeletonize`.
- Detection and visualization of skeleton **tips** (degree-1 pixels) and **junctions** (degree ≥ 3 pixels) using 3×3 convolution neighbor counting.
- Batch frame processing: iterate over all `.vtk` files in a directory and save a labeled skeleton plot per frame.
- Interactive frame-by-frame viewer built from a `.nd2` file directly, with optional mask overlay and keyboard navigation.

1.2 Dependencies

Package	Install command	Role
`numpy`	`pip install numpy`	Array operations and projection
`matplotlib`	`pip install matplotlib`	Plotting skeleton overlays and saving figures
`pyvista`	`pip install pyvista`	Loading `.vtk` files
`trimesh`	`pip install trimesh`	Voxelization of mesh geometry
`scikit-image`	`pip install scikit-image`	2D skeletonization
`scipy`	`pip install scipy`	Neighbor counting via 2D convolution
`nd2reader`	`pip install nd2reader`	Reading `.nd2` microscopy files
`tkinter`	(bundled with Python)	GUI window for the interactive viewer

Chapter 2

Notebook Cells Overview

All cells in this notebook are self-contained and can be run independently of one another. Each produces its own visualization.

2.1 Cell 1: Simple 2D Skeleton from a Single `.vtk`

Loads a single `.vtk` file, voxelizes it, collapses the 3D grid across the Z axis using a max projection, and applies 2D skeletonization. The binary mask and the skeleton are displayed overlaid in a Matplotlib figure.

Parameter	**Default**	**Description**
`mesh` (filepath)	(user-defined)	Path to the `.vtk` file to process.
`pitch`	`1`	Voxel size. Smaller values increase resolution at the cost of memory and speed.

2.2 Cell 2: Skeleton with Tips and Junctions Highlighted

Extends Cell 1 by detecting structural features after skeletonization. Tip pixels (exactly one skeleton neighbor) are displayed in lime green; junction pixels (three or more skeleton neighbors) are displayed in red. Both are overlaid on the skeleton via Matplotlib scatter plots.

The neighbor count is computed using a 3×3 convolution kernel:

$$K = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

A pixel is a **tip** if $K \star \text{skel} = 1$ and a **junction** if $K \star \text{skel} \geq 3$, where $\star$ denotes the convolution.

2.3 Cell 3: Batch Processing All Frames from `.vtk` Files

Iterates over all `.vtk` files in a specified input directory. For each file, the same pipeline as Cell 2 is applied: voxelize, project, skeletonize, detect features, and plot. Figures are saved to a `skeleton_plots/` output folder as numbered PNG files rather than displayed interactively.

Output folder: `skeleton_plots/` is created automatically in the working directory if it does not exist. Set `input_dir` to the folder containing the cleaned `.vtk` files produced by the `.vtk_Processing.ipynb` notebook.

2.4 Cell 4: Interactive Viewer from an `.nd2` File

Reads a raw `.nd2` microscopy file directly using `nd2reader`, performing Z-max projection and thresholding for each timepoint. The processed frames are then displayed in a Tkinter GUI window with Prev/Next buttons and keyboard arrow-key navigation. A checkbox toggles the binary mask overlay on top of the skeleton for each frame.

Parameter	Default	Description
`ND2_FILE`	(user-defined path)	Filepath to the `.nd2` microscopy file.
`VIEW_INDEX`	`0`	View/series index within the `.nd2` file.
`THRESH_FACTOR`	`0.3`	Fractional threshold applied to the normalized max projection. Pixels above this value are treated as foreground.

Chapter 3

Typical Workflow

(i) **Single frame check.** Run Cell 1 or Cell 2 on a representative `.vtk` frame to verify the `pitch` value produces a reasonable skeleton and that tip/junction detection looks correct.

(ii) **Batch export.** Run Cell 3 with `input_dir` pointing to the cleaned `.vtk` directory. Inspect the saved PNG files in `skeleton_plots/` to confirm consistent skeleton quality across all frames.

(iii) **ND2 inspection.** If working from raw `.nd2` data, run Cell 4 and use the interactive viewer to step through timepoints. Adjust `THRESH_FACTOR` if the skeleton is too sparse or too noisy.

Chapter 4

Limitations & Notes

Z Projection Loses Depth Information

Max projection collapses all Z slices into a single 2D image. Any three-dimensional structural information (e.g., branches that overlap when viewed from above) is irrecoverably lost. For full 3D analysis, use the `3D_Skeletonization.ipynb` notebook instead.

Pitch and Memory

Voxelization at small pitch values (e.g., `pitch = 0.5`) can produce very large 3D grids. For large meshes this may exhaust available RAM. The default `pitch = 1` is a practical starting point.

Intermediate STL file: Cell 1 and Cell 2 write a temporary `temp_mesh.stl` file to disk before loading with Trimesh. This file is overwritten on each run. To avoid collisions when running multiple cells simultaneously, rename it to a unique path per cell.

Chapter 5

Change Log

- **February 25, 2026:** File created from parts of earlier notebooks, consolidated in one place. (C. Richman)
- **May 20, 2026:** Tidied up for final repository. (C. Richman)